Introdução ao OpenChoreo (WSO2)

Last Updated: April 2026

Plataforma Moderna para Desenvolvedores em Kubernetes

Objetivo da apresentação

- Entender o que é o **OpenChoreo** e como ele funciona
- Conhecer os problemas que ele resolve no desenvolvimento moderno
- Compreender por que plataformas de engenharia moderna são necessárias
- Introduzir conceitos fundamentais do ecossistema cloud-native
- Preparar a base para entender arquitetura, módulos e componentes do **OpenChoreo**

O Desafio da Complexidade em Ambientes Cloud-Native

Como as empresas modernas desenvolvem software hoje

As organizações utilizam diversas tecnologias para construir aplicações modernas:

- **Kubernetes** para orquestração
- **Containers** para empacotamento de aplicações
- **Microservices** para modularidade
- **DevOps** e **CI/CD** para automação
- **Observabilidade** para monitoramento
- **Segurança** distribuída entre serviços

O problema: mais poder, mais complexidade

Apesar dos benefícios, esse ecossistema aumenta significativamente a complexidade operacional.

O Desafio da Complexidade em Ambientes Cloud-Native

Principais desafios enfrentados pelas equipes

- Kubernetes exige conhecimento avançado de infraestrutura
- Muitas ferramentas precisam ser integradas manualmente
- Desenvolvedores gastam tempo configurando ambiente em vez de programar
- Falta padronização entre projetos e times
- Governança e segurança tornam-se difíceis de manter
- Equipes de plataforma precisam construir e manter tudo do zero

Resultado: As empresas precisam de uma camada de abstração que simplifique Kubernetes sem perder flexibilidade.

É nesse contexto que o **OpenChoreo** surge, uma plataforma criada para reduzir complexidade e acelerar o desenvolvimento cloud-native.

Internal Developer Platform (IDP)

O **OpenChoreo** é uma plataforma open-source do tipo **Internal Developer Platform** (IDP). Ela funciona sobre **Kubernetes** e oferece uma experiência simplificada para desenvolvimento, implantação e operação de aplicações modernas.

Em outras palavras, o **OpenChoreo** transforma **Kubernetes** em uma plataforma mais acessível, organizada e amigável para desenvolvedores, com o objetivo de permitir que desenvolvedores foquem no código e nas regras de negócio, enquanto a plataforma gerencia a complexidade operacional.

O que o **OpenChoreo** oferece

- **Plataforma self-service** para equipes de desenvolvimento
- **Deploy automatizado** de aplicações
- Fluxo baseado em **GitOps**
- **Observabilidade** integrada
- Portal **centralizado** para desenvolvedores
- **Governança** e padronização de ambientes
- **Segurança** integrada desde o início

IDP: Simplificando o Desenvolvimento

Uma **Internal Developer Platform (IDP)** é uma camada de abstração criada para conectar:

- Infraestrutura complexa
- Equipes de desenvolvimento

Ela oferece ferramentas, automação e padrões que simplificam o uso da plataforma.

Objetivo principal: Reduzir a complexidade operacional e permitir que desenvolvedores foquem na construção de aplicações.

Analogia simples

- Kubernetes é o motor do carro
- A IDP é o painel inteligente que facilita dirigir
- OpenChoreo atua como essa camada de simplificação

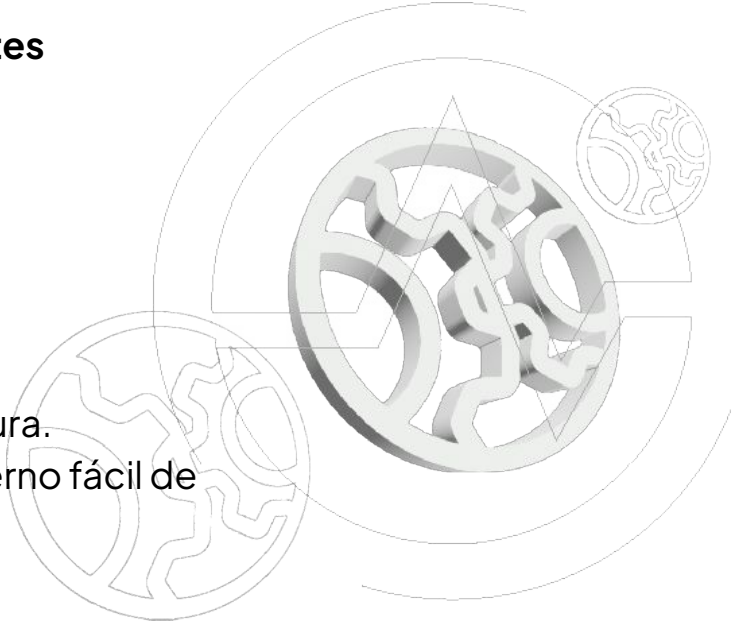
IDP: Simplificando o Desenvolvimento

Benefícios de uma IDP

- Menos arquivos YAML e configuração manual
- Redução da curva de aprendizado em **Kubernetes**
- Maior produtividade das equipes
- Padronização de ambientes e processos
- Governança centralizada
- Melhor experiência para desenvolvedores

Por que isso importa?

Sem uma IDP, cada time precisa aprender infraestrutura.
Com uma IDP, a infraestrutura se torna um serviço interno fácil de consumir.



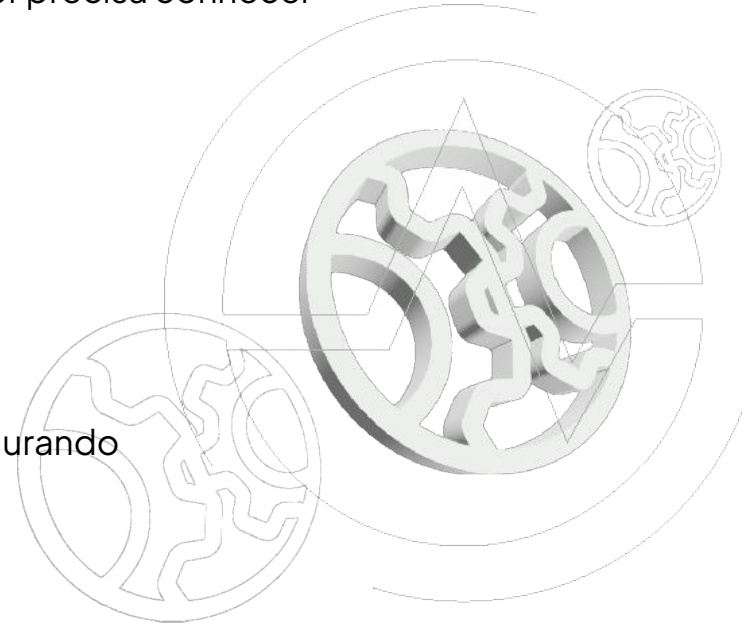
Como o OpenChoreo Reduz a Complexidade

Antes do OpenChoreo

Em ambientes cloud-native tradicionais, o desenvolvedor precisa conhecer diversas áreas além do código:

- Kubernetes e orquestração
- Configuração de networking
- Pipelines de CI/CD
- Observabilidade e monitoramento
- Segurança e permissões
- Estratégias de deploy e rollback

O impacto disso: Grande parte do tempo é gasto configurando infraestrutura em vez de desenvolver funcionalidades.



Como o OpenChoreo Reduz a Complexidade

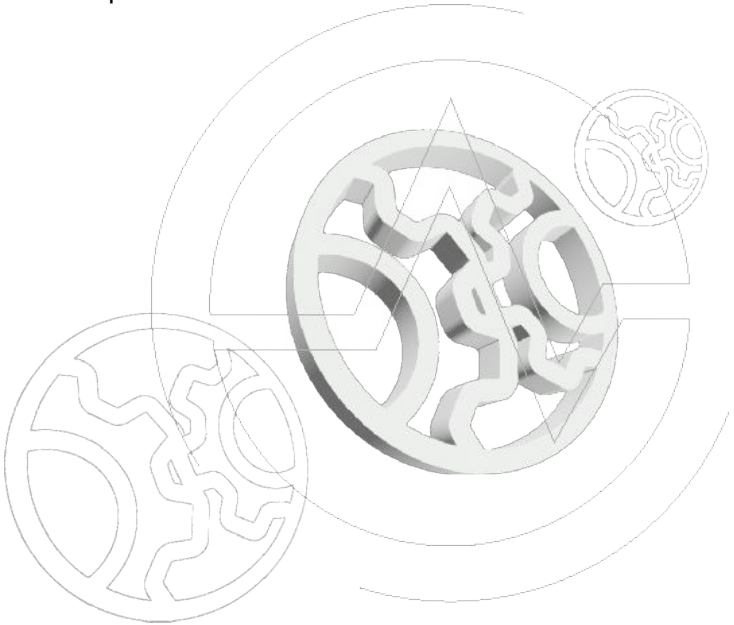
Depois do OpenChoreo

A plataforma abstrai a complexidade operacional, e o desenvolvedor passa a focar principalmente em:

- Código da aplicação
- APIs e integrações
- Regras de negócio
- Entrega de valor ao usuário

Ganhos para equipes e organizações

- Redução significativa da complexidade
- Padronização entre projetos
- Entregas mais rápidas e previsíveis
- Menor curva de aprendizado
- Melhor experiência para desenvolvedores
- Maior eficiência operacional



Onde o OpenChoreo Pode Ser Utilizado

1. Plataformas Internas de Desenvolvimento

- Criar um portal centralizado para desenvolvedores acessarem ambientes, serviços, deploys e observabilidade.
- **Benefício:** Padronização e experiência self-service.

2. Arquiteturas Baseadas em Microservices

- Gerenciar dezenas ou centenas de serviços distribuídos em Kubernetes.
- **Benefício:** Organização, escalabilidade e controle operacional.

3. APIs Corporativas

- Publicar APIs de forma segura, governada e padronizada.
- **Benefício:** Controle de exposição, segurança e integração entre sistemas.

Onde o OpenChoreo Pode Ser Utilizado

4. Times DevOps e Platform Engineering

- Criar uma camada de abstração para reduzir a complexidade da infraestrutura.
- **Benefício:** Padronizar deploys, pipelines e ambientes.

5. Aplicações Cloud-Native

- Executar aplicações modernas diretamente sobre Kubernetes com menor esforço operacional.
- **Benefício:** Deploy simplificado e operação consistente.

Resumo

- **OpenChoreo** é ideal para organizações que precisam escalar desenvolvimento sem aumentar complexidade operacional.

Ferramentas Fundamentais do Ecossistema Kubernetes

Docker

- **O que é:** Plataforma de containers utilizada para empacotar aplicações.
- **O que faz:** Agrupa aplicação, bibliotecas e dependências em uma única imagem executável.
- **Por que usar:** Permite que a aplicação funcione da mesma forma em qualquer ambiente.

k3d

- **O que é:** Ferramenta que executa clusters Kubernetes leves dentro do Docker.
- **O que faz:** Cria rapidamente ambientes Kubernetes locais para desenvolvimento e testes.
- **Por que usar:** Ideal para laboratório, aprendizado e simulação de ambientes reais.

Ferramentas Fundamentais do Ecossistema Kubernetes

kubectl

- **O que é:** CLI oficial do Kubernetes.
- **O que faz:** Permite administrar clusters Kubernetes via linha de comando.
- **Por que usar:** Utilizado para deploy, configuração, logs, monitoramento e troubleshooting.

Helm

- **O que é:** Gerenciador de pacotes para Kubernetes.
- **O que faz:** Automatiza instalação e configuração de aplicações complexas.
- **Por que usar:** Reduz a quantidade de YAML manual e facilita deploys repetíveis.

Por que essas ferramentas são importantes?

Elas formam a base tecnológica utilizada pelo OpenChoreo para construir uma plataforma cloud-native moderna.

Tecnologias que Formam a Base do OpenChoreo

OpenChoreo não substitui Kubernetes, ele organiza e integra o ecossistema cloud-native

- A plataforma reúne múltiplas tecnologias modernas em uma experiência unificada.

Kubernetes

- Responsável por executar e orquestrar aplicações em containers.
- **Função:** Base de execução da plataforma.

GitOps

- Modelo operacional baseado em Git como fonte de verdade.
- **Função:** Controlar configurações e deploys de forma declarativa.

Tecnologias que Formam a Base do OpenChoreo

Backstage

- Framework utilizado para experiência de portal de desenvolvedor.
- **Função:** Centralizar serviços, documentação e recursos internos.

Observability Stack

- Conjunto de ferramentas para monitoramento.
- **Função:** Fornecer logs, métricas e tracing.

API Gateway

- Camada responsável por exposição de APIs.
- **Função:** Controlar acesso, segurança e roteamento.

Tecnologias que Formam a Base do OpenChoreo

Networking Policies

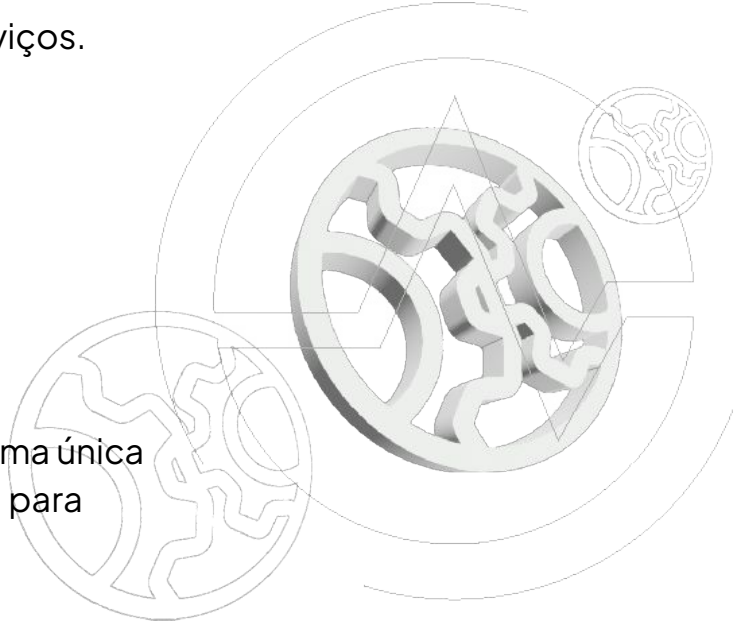
- Políticas de comunicação entre workloads.
- **Função:** Garantir isolamento e segurança entre serviços.

CI/CD Pipelines

- Fluxos automatizados de build e deploy.
- **Função:** Automatizar entrega contínua.

Resultado

- **OpenChoreo** integra todas essas tecnologias em uma única plataforma, reduzindo a complexidade operacional para desenvolvedores e equipes de plataforma.

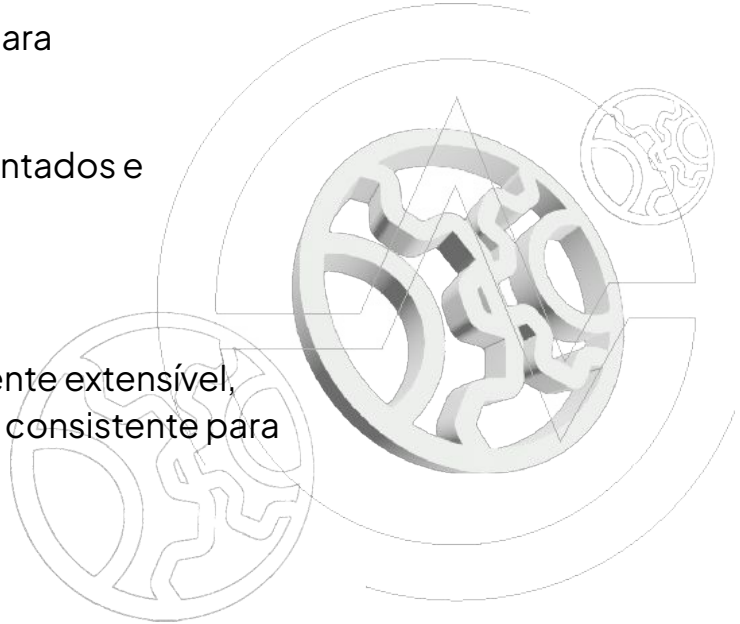


Recursos de Plataforma: Extensibilidade

OpenChoreo utiliza recursos de plataforma para padronizar e automatizar comportamentos

Esses recursos permitem criar experiências reutilizáveis para desenvolvedores e equipes de plataforma.

Eles definem como componentes são construídos, implantados e operados.



Resultado: Esses recursos tornam o OpenChoreo altamente extensível, reutilizável e governado, permitindo criar uma plataforma consistente para múltiplos times.

Recursos de Plataforma: Extensibilidade

Workflow

- **O que é:** Define um fluxo reutilizável de execução.
- **Função:** Executar etapas automatizadas como build, teste e deploy.

Exemplo: Pipeline padrão para aplicações backend.

Trait

- **O que é:** Configuração adicional aplicada a um componente.
- **Função:** Adicionar capacidades sem alterar a aplicação.
- **Exemplos:**
  - Autoscaling
  - Ingress
  - Observabilidade
  - Configuração de recursos

Recursos de Plataforma: Extensibilidade

ComponentType

- **O que é:** Modelo reutilizável para criação de componentes.
- **Função:** Definir como determinado tipo de aplicação deve funcionar.
- **Exemplos:**
  - Serviço REST
  - Worker
  - Frontend
  - Job agendado

ClusterWorkflow

- **O que é:** Workflow disponível globalmente para um cluster.
- **Função:** Padronizar pipelines para múltiplos projetos.
- **Benefício:** Governança centralizada.

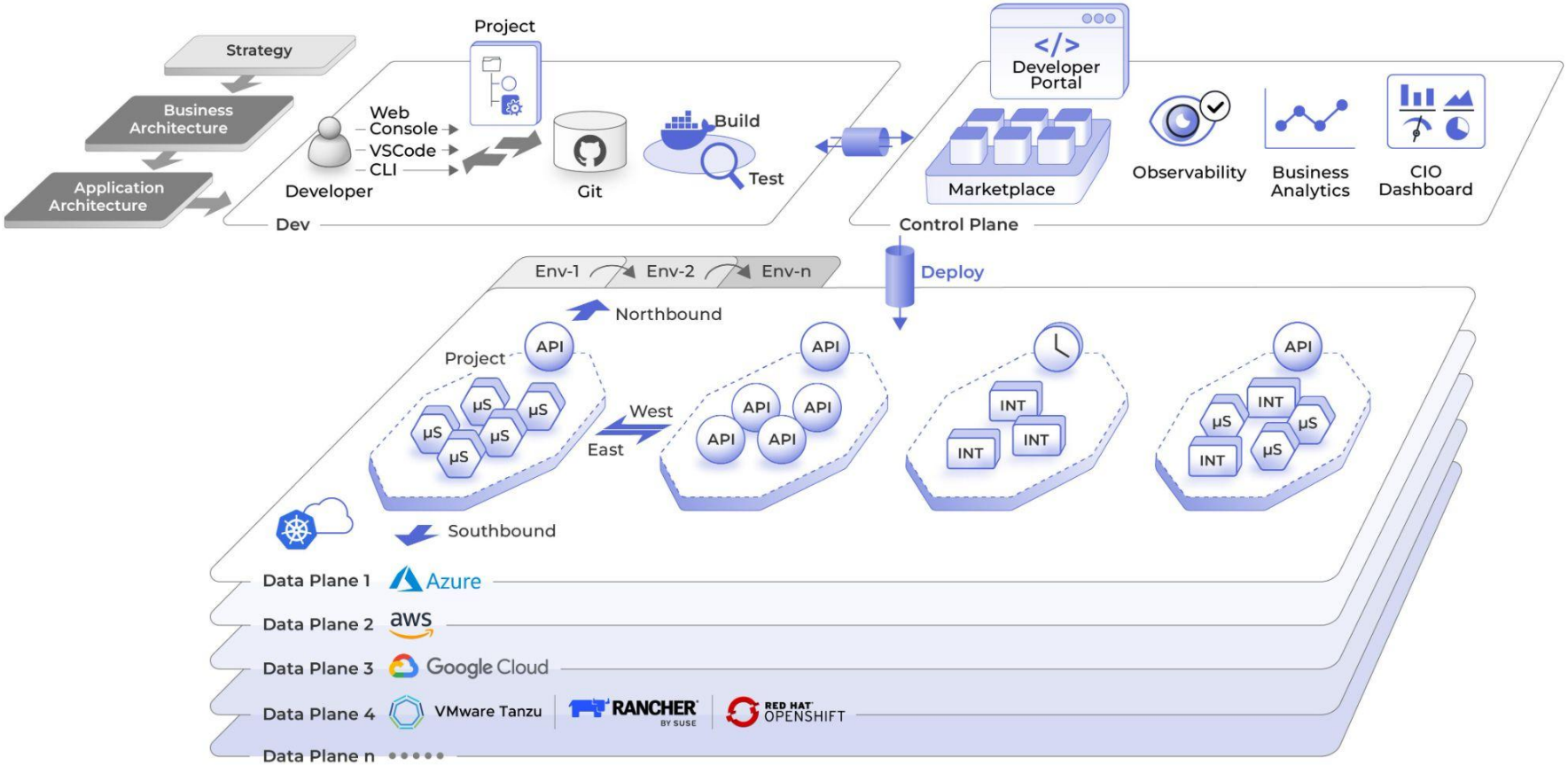
Recursos de Plataforma: Extensibilidade

ClusterTrait

- **O que é:** Trait compartilhado em nível de cluster.
- **Função:** Aplicar configurações reutilizáveis para todos os componentes.
- **Benefício:** Consistência operacional.

ClusterComponentType

- **O que é:** ComponentType definido globalmente para o cluster.
- **Função:** Disponibilizar templates padrão para toda a organização.
- **Benefício:** Padronização entre equipes.


Strategy
Business Architecture
Application Architecture
Project
Developer
Web Console
VSCode
CLI
Git
Build
Test
Dev
Developer Portal
Marketplace
Observability
Business Analytics
CIO Dashboard
Control Plane
Deploy
Env-1
Env-2
Env-n
Northbound
Project
API
μS
West
East
INT
Southbound
Data Plane 1
Azure
Data Plane 2
aws
Data Plane 3
Google Cloud
Data Plane 4
VMware Tanzu
RANCHER BY SUSE
RED HAT OPENSHIFT
Data Plane n

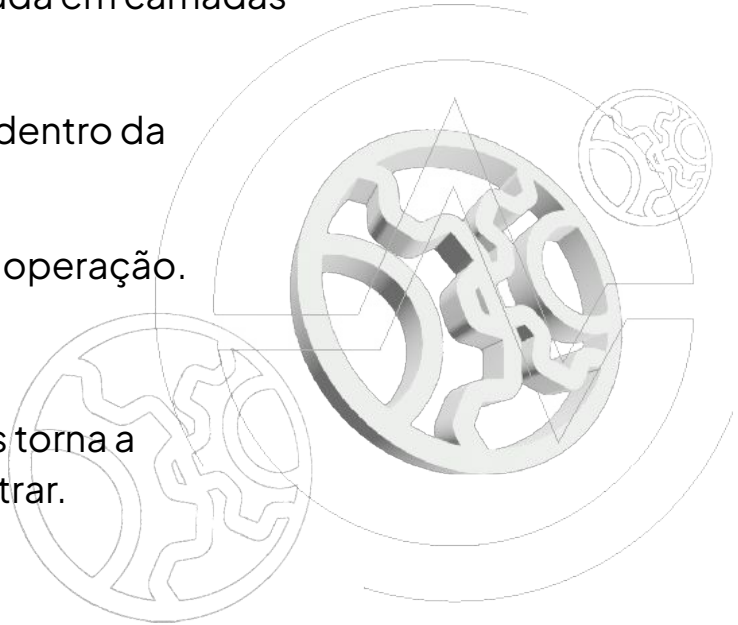
Arquitetura Baseada em “Planes”

Como o OpenChoreo organiza sua arquitetura

OpenChoreo utiliza uma arquitetura distribuída baseada em camadas chamadas de **Planes**.

Cada plane possui uma responsabilidade específica dentro da plataforma.

Essa separação facilita escalabilidade, governança e operação.



Benefício da Arquitetura: Separar responsabilidades torna a plataforma mais modular, escalável e fácil de administrar.

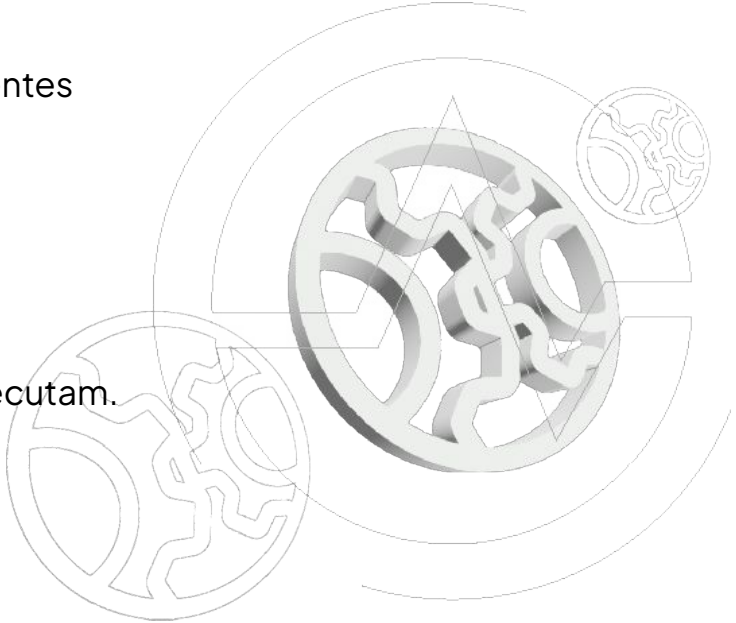
Arquitetura Baseada em “Planes”

Control Plane

- **Função:** É o “cérebro” da plataforma.
- **Responsabilidades:**
  - Gerenciar organizações, projetos e componentes
  - Orquestrar recursos da plataforma
  - Controlar configurações e políticas
  - Coordenar deploys e ambientes

Data Plane

- **Função:** É o local onde as aplicações realmente executam.
- **Responsabilidades:**
  - Hospedar workloads Kubernetes
  - Executar serviços e APIs
  - Gerenciar runtime das aplicações



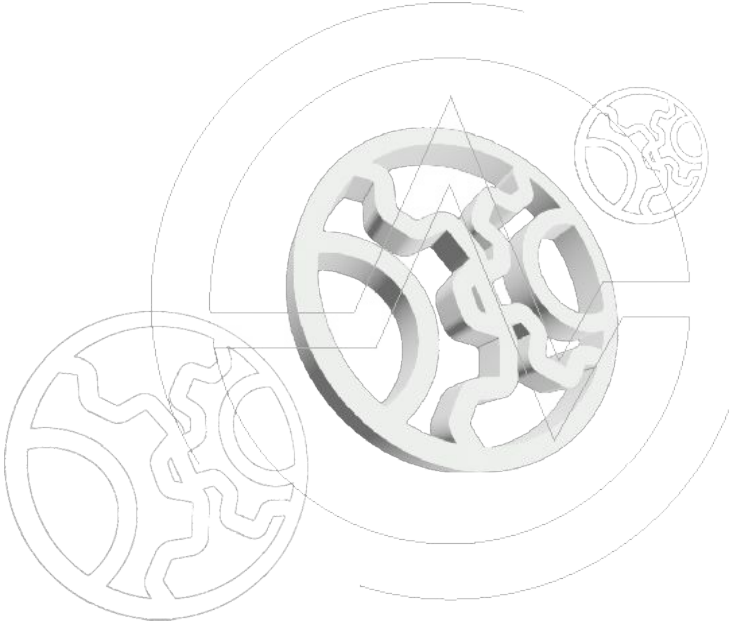
Arquitetura Baseada em “Planes”

Workflow Plane

- **Função:** Responsável pelos fluxos de automação.
- **Responsabilidades:**
  - Executar pipelines CI/CD
  - Automatizar build e deploy
  - Integrar GitOps e processos de entrega

Observability Plane

- **Função:** Fornecer visibilidade operacional.
- **Responsabilidades:**
  - Coletar logs
  - Monitorar métricas
  - Realizar tracing distribuído
  - Facilitar troubleshooting



Organização Lógica da Plataforma OpenChoreo

Como o OpenChoreo organiza aplicações e equipes

- **OpenChoreo** utiliza uma estrutura lógica para organizar projetos, serviços e ambientes.
- Essa organização ajuda a padronizar desenvolvimento, deploy e governança.

Organization

- **O que é:** Representa uma empresa, time ou domínio organizacional.
- **Função:** Agrupar usuários, projetos e políticas.

Project

- **O que é:** Uma unidade lógica de trabalho dentro da organização.
- **Função:** Agrupar componentes relacionados a uma mesma aplicação ou solução.

Organização Lógica da Plataforma OpenChoreo

Component

- **O que é:** A menor unidade implantável da plataforma.
- **Função:** Representar um serviço, API, frontend, job ou worker.

Endpoint

- **O que é:** A interface exposta por um componente.
- **Função:** Permitir acesso externo ou interno ao serviço.
- **Exemplos:**
  - API REST
  - Endpoint HTTP
  - Serviço interno

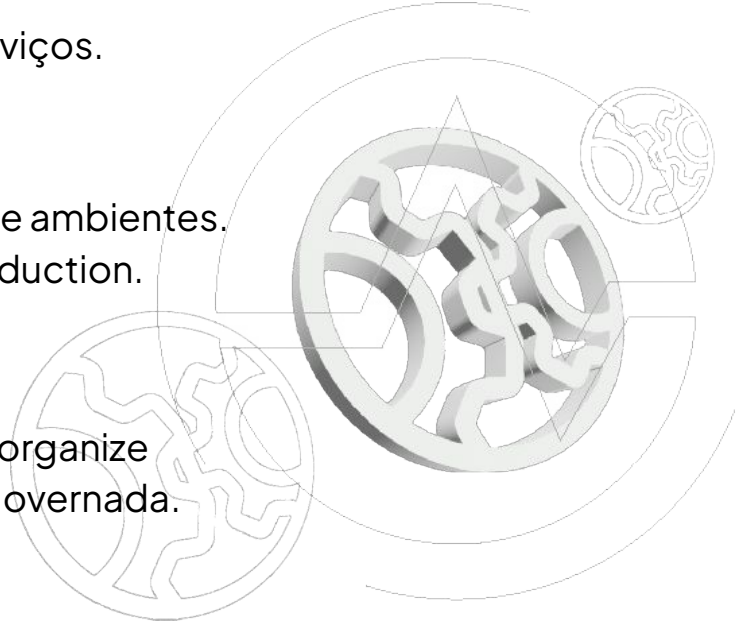
Organização Lógica da Plataforma OpenChoreo

Connection

- **O que é:** Relacionamento entre componentes.
- **Função:** Permitir comunicação segura entre serviços.

Deployment Pipeline

- **O que é:** Fluxo automatizado de promoção entre ambientes.
- **Função:** Mover aplicações entre Dev, Test e Production.



Resultado: Essa estrutura permite que OpenChoreo organize aplicações cloud-native de forma clara, escalável e governada.

Fluxo Simplificado de Desenvolvimento e Deploy

O **OpenChoreo** automatiza o ciclo de vida completo de uma aplicação cloud-native.

1. Desenvolvedor envia o código

- O código é conectado à plataforma através de um repositório Git.

2. OpenChoreo detecta o tipo de aplicação

- A plataforma identifica automaticamente:
  - Linguagem utilizada
  - Framework
  - Tipo de workload
  - Estratégia de build

Fluxo Simplificado de Desenvolvimento e Deploy

3. Build automático

- OpenChoreo gera a imagem da aplicação utilizando pipelines integrados.

4. Deploy automatizado

- A aplicação é implantada automaticamente em Kubernetes.

5. Observabilidade integrada

- A plataforma habilita monitoramento sem configuração manual.
- Inclui:
  - Logs
  - Métricas
  - Tracing distribuído

Fluxo Simplificado de Desenvolvimento e Deploy

6. Exposição de APIs e Endpoints

- A aplicação pode ser publicada com acesso controlado e seguro.

Resultado

- Menos esforço operacional
- Menos configuração manual
- Maior velocidade de entrega
- Mais foco no desenvolvimento e no negócio

Objetivo: Transformar o processo de deploy em uma experiência simples e repetível.

Thanks!

wso2.com